

Instructions for Authors of SBC Conferences

Papers and Abstracts

Luciana P. Nedel¹, Rafael H. Bordini², Flávio Rech Wagner¹, Jomi F. Hübner³

¹Instituto de Informática – Universidade Federal do Rio Grande do Sul (UFRGS)
Caixa Postal 15.064 – 91.501-970 – Porto Alegre – RS – Brazil

²Department of Computer Science – University of Durham
Durham, U.K.

³Departamento de Sistemas e Computação
Universidade Regional de Blumenau (FURB) – Blumenau, SC – Brazil

{nedel, flavio}@inf.ufrgs.br, R.Bordini@durham.ac.uk, jomi@inf.furb.br

1. Introdução

O tema deste trabalho consiste na análise empírica de algoritmos de roteamento e busca de caminhos mínimos (*shortest path*) em grafos. Dentro desse contexto, o problema central a ser resolvido é a determinação da rota de menor custo entre um vértice de origem e todos os demais vértices presentes em uma rede, uma tarefa essencial para a comunicação e atualização de tabelas de roteamento intra-domínio.

O objetivo do experimento é investigar empiricamente o comportamento prático desses algoritmos e relacioná-lo à sua análise assintótica. Para isso, o tamanho da entrada será aumentado de forma sistemática com o intuito de medir o tempo de execução de cada algoritmo, comparando o crescimento observado na prática com a complexidade esperada.

2. Algoritmos e fontes

Para a resolução do problema de roteamento e busca de caminhos mínimos, utilizamos como base o artigo “A Study on Contrast and Comparison between Bellman-Ford algorithm and Dijkstra’s algorithm” [Thippeswamy et al. 2010]. Este trabalho explora e compara dois dos algoritmos mais populares empregados na atualização de tabelas de roteamento intra-domínio: o Algoritmo de Bellman-Ford e o Algoritmo de Dijkstra.

A origem do código adotado em nosso experimento derivou diretamente dessa literatura. A implementação foi construída a partir da lógica estrutural e dos pseudocódigos fornecidos pelos autores no artigo, que detalham os procedimentos fundamentais passo a passo para a execução de ambos os algoritmos, sem a necessidade de importar códigos de repositórios externos.

3. Metodologia

/* Aqui é sua parte Letícia */

Para a medição e comparação dos dois algoritmos, foi implementado um header em C++ responsável por medir o tempo ao executar o algoritmo (*ms*), e ao terminar a execução do algoritmo (*ms*), para o Dijkstra e o Bellman-Ford, geradas e registradas em

um arquivo em formato *Comma-Separated Values (csv)*, usados para formar os gráficos futuramente. Como método de testes, utilizou-se tamanhos de entradas como 100, 1.000, 10.000, 1.000.000 e 10.000.000 cada uma dessas entradas foram testadas 32 vezes e calculado uma média aritmética, dividindo pela quantidade de testes realizados.

4. Análise teórica

Para compreender as diferenças de desempenho que serão observadas na prática, é fundamental estabelecer primeiro a base teórica de cada abordagem. A complexidade assintótica dos algoritmos de Dijkstra e Bellman-Ford decorre diretamente da estrutura de seus laços de repetição e do custo de suas operações principais, conforme detalhado a seguir.

No caso do algoritmo de Dijkstra, como o algoritmo visita cada vértice e avalia suas respectivas conexões, o tempo de execução resulta na equação $O(|V|^2 + |E|)$, que pode ser assintoticamente simplificada para $O(|V|^2)$, onde $|V|$ representa o número de vértices e $|E|$ o número de arestas [Thippeswamy et al. 2010]. Em um cenário de rede hipotético onde o número de nós (n) é equivalente ao número de arestas (m), o tempo de processamento estimado cresce proporcionalmente a $2n^2$, confirmando o comportamento de ordem quadrática [Thippeswamy et al. 2010].

Por outro lado, o algoritmo de Bellman-Ford adota uma estratégia estruturalmente distinta. Para assegurar que as distâncias mínimas sejam propagadas com precisão por toda a rede, o algoritmo executa um laço de relaxamento sobre o conjunto completo de arestas exatamente $|V| - 1$ vezes [Thippeswamy et al. 2010]. Devido a essa varredura exaustiva, o tempo total é regido pela multiplicação da quantidade de vértices pela de arestas, estabelecendo uma complexidade assintótica geral de $O(|V| \times |E|)$ [Thippeswamy et al. 2010].

Para o cenário onde o número de arestas (m) se iguala ao de nós (n), a complexidade computacional do Bellman-Ford é matematicamente descrita pela função $27n^2 + 28n + 3$ [Thippeswamy et al. 2010]. O termo dominante dessa equação determina uma ordem assintótica $O(n^2)$, que pode ser simplificada para a proporção de $27n^2$ [Thippeswamy et al. 2010]. Embora a análise indique um crescimento assintótico quadrático para ambos os algoritmos nessas condições, o coeficiente substancialmente maior no Bellman-Ford fornece a justificativa teórica para o seu tempo de execução prático ser consideravelmente superior ao de Dijkstra [Thippeswamy et al. 2010].

5. Resultados

A presente seção apresenta os resultados empíricos coletados a partir dos algoritmos para fins de comparação entre seus tempos de execução com base no tamanho de suas entradas, resumidos na Tabela 1.

Para melhor compreensão comparativa, os dados apresentados foram plotados na Figura 1, demonstrando visualmente o distanciamento entre as curvas de tempo de execução à medida que o tamanho da entrada aumenta.

A partir da leitura da Figura 1, observa-se que [TODO: DESCRIVER COMPORTAMENTO COM DADOS FINAIS].

Table 1. Tempos de execução em milissegundos para grafos de tamanho n .

Tamanho da Entrada (n)	Dijkstra (ms)	Bellman-Ford (ms)
100	0.5	1.2
200	2.1	5.8
300	4.8	15.2

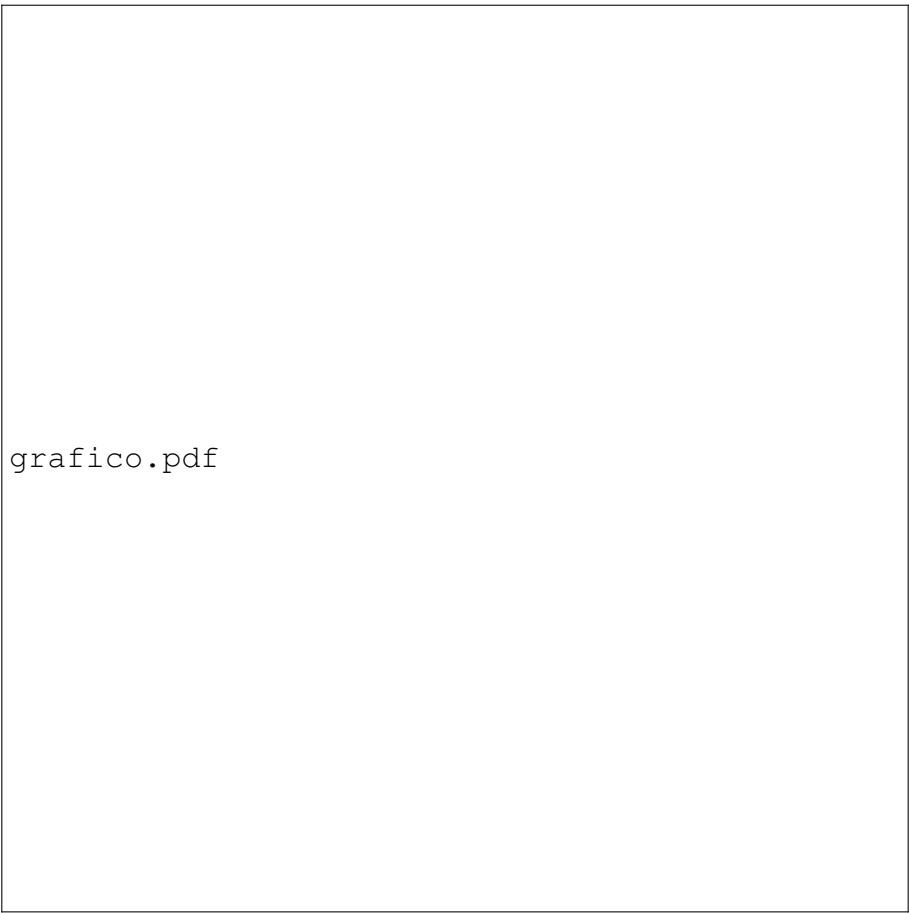


Figure 1. Comparação do tempo de execução empírico entre Dijkstra e Bellman-Ford.

6. Discussão

Sob a ótica dos resultados coletados, observa-se uma eficiência notavelmente superior do algoritmo de Dijkstra em comparação ao algoritmo de Bellman-Ford no que diz respeito à busca de caminhos em roteamento intra-domínio. Os dados empíricos apresentados na Seção 5 corroboram a análise teórica de [Thippeswamy et al. 2010], fortalecendo a premissa de que, embora ambos resolvam o problema de busca do caminho mais curto a partir de uma única origem, o algoritmo de Dijkstra desempenha este papel com menor custo computacional.

Isto ocorre pois o algoritmo de Dijkstra se baseia em uma abordagem gulosa, ou seja, a cada passo, é realizada uma escolha localmente ótima, subsequentemente não reconsiderando escolhas passadas. Para tal, o algoritmo seleciona o nó de peso mínimo que ainda não foi processado para realizar o relaxamento das arestas. Isto torna o algoritmo mais performático caso haja premissas estruturais baseadas na existência exclusiva de pesos positivos. Dessa forma, sua complexidade reflete a abordagem, configurando-se como $\mathcal{O}(|V|^2 + |E|)$, simplificada para $\mathcal{O}(|V|^2)$.

Em contrapartida, o algoritmo de Bellman-Ford não adota uma estratégia gulosa. Em vez disso, ele executa um processo exaustivo de relaxamento de todas as arestas por exatas $|V| - 1$ vezes. Essa repetição garante a correta propagação das distâncias mínimas por todo o grafo e confere ao algoritmo a capacidade de processar arestas com pesos negativos. No entanto, o custo computacional dessa flexibilidade reflete-se em sua complexidade teórica de $\mathcal{O}(|V| \times |E|)$, o que acarreta um aumento drástico no tempo de execução, especialmente à medida que o grafo se torna mais denso, com um maior $|E|$.

Vale destacar que, para grafos de tamanho pequeno, o tempo de execução de ambos os algoritmos é virtualmente idêntico, como observado na Figura 1. Com um tamanho de entrada n suficientemente pequeno, o tempo de alocação inicial das estruturas e as constantes ocultas da notação dominam o tempo total, atenuando a diferença entre o crescimento de $|V|^2$ e $|V| \times |E|$. Diante dos resultados, o comportamento assintótico torna-se substancial a partir de [TODO: qual tamanho de entrada define o limiar de diferença], demonstrando o crescimento abrupto do algoritmo de Bellman-Ford.

7. Conclusão

A análise empírica realizada neste trabalho confirma a superioridade em eficiência do algoritmo de Dijkstra frente ao Bellman-Ford na busca do caminho mais curto para roteamento intra-domínio. A abordagem gulosa poupa processamento em comparação à varredura exaustiva do Bellman-Ford, consolidando o Dijkstra como a escolha ideal para grafos com pesos estritamente positivos por apresentar tempos de execução consistentemente menores.

Ainda assim, o algoritmo de Bellman-Ford mantém sua relevância fundamental em cenários de roteamento que admitem arestas de peso negativo. Seu elevado custo computacional justifica-se pela capacidade de encontrar caminhos em redes complexas de forma confiável. É imperativo ressaltar, contudo, que caso existam ciclos negativos alcançáveis a partir da origem, nenhum dos algoritmos é capaz de computar o caminho mais curto, visto que o peso total decresce infinitamente a cada travessia no ciclo.

Por fim, cabe destacar que os tempos absolutos aferidos nesta análise são

intrínsecos ao *hardware* e ao sistema operacional utilizados no experimento. Como direcionamento para trabalhos futuros, sugere-se a execução de testes empíricos em grafos contendo arestas negativas, a fim de demonstrar na prática a falha inerente do algoritmo de Dijkstra e a resiliência do Bellman-Ford diante de tal cenário.

8. General Information

All full papers and posters (short papers) submitted to some SBC conference, including any supporting documents, should be written in English or in Portuguese. The format paper should be A4 with single column, 3.5 cm for upper margin, 2.5 cm for bottom margin and 3.0 cm for lateral margins, without headers or footers. The main font must be Times, 12 point nominal size, with 6 points of space before each paragraph. Page numbers must be suppressed.

Full papers must respect the page limits defined by the conference. Conferences that publish just abstracts ask for **one**-page texts.

9. First Page

The first page must display the paper title, the name and address of the authors, the abstract in English and “resumo” in Portuguese (“resumos” are required only for papers written in Portuguese). The title must be centered over the whole page, in 16 point boldface font and with 12 points of space before itself. Author names must be centered in 12 point font, bold, all of them disposed in the same line, separated by commas and with 12 points of space after the title. Addresses must be centered in 12 point font, also with 12 points of space after the authors’ names. E-mail addresses should be written using font Courier New, 10 point nominal size, with 6 points of space before and 6 points of space after.

The abstract and “resumo” (if is the case) must be in 12 point Times font, indented 0.8cm on both sides. The word **Abstract** and **Resumo**, should be written in boldface and must precede the text.

10. CD-ROMs and Printed Proceedings

In some conferences, the papers are published on CD-ROM while only the abstract is published in the printed Proceedings. In this case, authors are invited to prepare two final versions of the paper. One, complete, to be published on the CD and the other, containing only the first page, with abstract and “resumo” (for papers in Portuguese).

11. Sections and Paragraphs

Section titles must be in boldface, 13pt, flush left. There should be an extra 12 pt of space before each title. Section numbering is optional. The first paragraph of each section should not be indented, while the first lines of subsequent paragraphs should be indented by 1.27 cm.

11.1. Subsections

The subsection titles must be in boldface, 12pt, flush left.



Figure 2. A typical figure

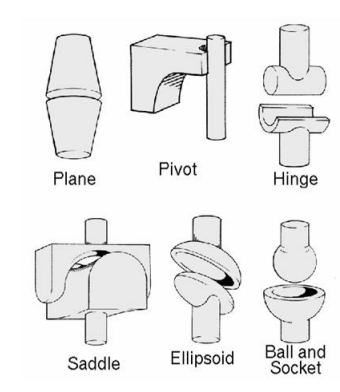


Figure 3. This figure is an example of a figure caption taking more than one line and justified considering margins mentioned in Section 12.

12. Figures and Captions

Figure and table captions should be centered if less than one line (Figure 2), otherwise justified and indented by 0.8cm on both margins, as shown in Figure 3. The caption font must be Helvetica, 10 point, boldface, with 6 points of space before and after each caption.

In tables, try to avoid the use of colored or shaded backgrounds, and avoid thick, doubled, or unnecessary framing lines. When reporting empirical data, do not use more decimal digits than warranted by their precision and reproducibility. Table caption must be placed before the table (see Table 1) and the font used must also be Helvetica, 10 point, boldface, with 6 points of space before and after each caption.

13. Images

All images and illustrations should be in black-and-white, or gray tones, excepting for the papers that will be electronically available (on CD-ROMs, internet, etc.). The image resolution on paper should be about 600 dpi for black-and-white images, and 150-300 dpi for grayscale images. Do not include images with excessive resolution, as they may take hours to print, without any visible difference in the result.

Table 2. Variables to be considered on the evaluation of interaction techniques

	Chessboard top view	Chessboard perspective view
Selection with side movements	6.02 $\pm$ 5.22	7.01 $\pm$ 6.84
Selection with in- depth movements	6.29 $\pm$ 4.99	12.22 $\pm$ 11.33
Manipulation with side movements	4.66 $\pm$ 4.94	3.47 $\pm$ 2.20
Manipulation with in- depth movements	5.71 $\pm$ 4.55	5.37 $\pm$ 3.28

14. References

Bibliographic references must be unambiguous and uniform. We recommend giving the author names references in brackets, e.g. [Knuth 1984], [Boulic and Renault 1991], and [Smith and Jones 1999].

The references must be listed using 12 point font size, with 6 points of space before each reference. The first line of each reference should not be indented, while the subsequent should be indented by 0.5 cm.

References

- Boulic, R. and Renault, O. (1991). 3d hierarchies for animation. In Magnenat-Thalmann, N. and Thalmann, D., editors, *New Trends in Animation and Visualization*. John Wiley & Sons Ltd.
- Knuth, D. E. (1984). *The T_EX Book*. Addison-Wesley, 15th edition.
- Smith, A. and Jones, B. (1999). On the complexity of computing. In Smith-Jones, A. B., editor, *Advances in Computer Science*, pages 555–566. Publishing Press.
- Thippeswamy, K., Hanumanthappa, J., and Manjaiah, D. H. (2010). A study on contrast and comparison between bellman-ford algorithm and dijkstra’s algorithm. *Department of Information Science and Engg, RLJIT*.